Queries and Interpretation

1. How many reservations and planned room-nights are currently in each reservation status?

SQL

```
SELECT

    r.reservationStatus,

    COUNT(*) AS reservation_count,

    SUM(julianday(r.plannedCheckOutDate) -
julianday(r.plannedCheckInDate)) AS planned_room_nights,

    SUM(rp.numGuests) AS expected_guests

FROM reservation r

JOIN reservation_preference rp ON rp.reservationId = r.reservationId

GROUP BY r.reservationStatus

ORDER BY reservation_count DESC, r.reservationStatus;
```

OUTPUT

BOOKED|3|8.0|4

CHECKED_IN|3|7.0|5

2. On which check-in dates is guest arrival demand the highest?

SQL

```
SELECT

    r.plannedCheckInDate,

    COUNT(*) AS arrivals,

    SUM(rp.numGuests) AS expected_guests,

    SUM(julianday(r.plannedCheckOutDate) -
```

```
julianday(r.plannedCheckInDate)) AS expected_room_nights
FROM reservation r
JOIN reservation_preference rp ON rp.reservationId = r.reservationId
GROUP BY r.plannedCheckInDate
ORDER BY r.plannedCheckInDate;
```

OUTPUT

2026-03-31|1|2|2.0

2026-04-01|2|3|5.0

2026-04-02|1|2|3.0

2026-04-03|1|1|3.0

2026-04-05|1|1|2.0

3. How is guest-room inventory distributed across wings, and how convenient is each wing for parking or pool access?

SQL

```
SELECT
    b.buildingName,
    w.wingCode,
    COUNT(*) AS room_count,
    SUM(CASE WHEN w.nearParking = 1 THEN 1 ELSE 0 END) AS
near_parking_rooms,
    SUM(CASE WHEN w.nearIndoorPool = 1 THEN 1 ELSE 0 END) AS
near_indoor_pool_rooms,
    SUM(CASE WHEN w.nearOutdoorPool = 1 THEN 1 ELSE 0 END) AS
near_outdoor_pool_rooms
FROM room r
JOIN level l ON l.levelId = r.levelId
JOIN wing w ON w.wingId = l.wingId
```

```
JOIN building b ON b.buildingId = w.buildingId
GROUP BY b.buildingName, w.wingCode
ORDER BY room_count DESC, b.buildingName, w.wingCode;
```

OUTPUT

Main Building|A|10|10|10|0

Main Building|B|10|0|0|10

4. Which wing offers the highest average nightly rate and average sleeping capacity for guest rooms?

SQL

```
SELECT
    b.buildingName,
    w.wingCode,
    ROUND(AVG(rc.baseRate), 2) AS avg_nightly_rate,
    ROUND(AVG(rc.capacity), 2) AS avg_sleeping_capacity,
    COUNT(*) AS sleeping_rooms
FROM room_capability rc
JOIN capability_type ct ON ct.capabilityTypeId = rc.capabilityTypeId
JOIN room r ON r.roomId = rc.roomId
JOIN level l ON l.levelId = r.levelId
JOIN wing w ON w.wingId = l.wingId
JOIN building b ON b.buildingId = w.buildingId
WHERE ct.capabilityCode = 'SLEEPING'
GROUP BY b.buildingName, w.wingCode
ORDER BY avg_nightly_rate DESC, avg_sleeping_capacity DESC;
```

OUTPUT

Main Building|A|191.88|2.25|8

Main Building|B|176.67|1.83|6

5. What bed sizes are most common across guest rooms?

SQL

```
SELECT
    bt.size AS bed_size,
    COUNT(DISTINCT rb.roomId) AS rooms_with_bed_type,
    SUM(rb.quantity) AS total_beds,
FROM room_bed rb
JOIN bed_type bt ON bt.bedTypeId = rb.bedTypeId
JOIN room r ON r.roomId = rb.roomId
GROUP BY bt.size
ORDER BY rooms_with_bed_type DESC, total_beds DESC, bed_size;
```

OUTPUT

KING|7|7

QUEEN|6|7

TWIN|4|6

ROLLAWAY|1|1

6. What room preference patterns appear most often among active reservations?

SQL

```
SELECT
    CASE
        WHEN rp.smokingPref = 0 THEN 'Non-smoking'
        WHEN rp.smokingPref = 1 THEN 'Smoking'
        ELSE 'No smoking preference'
    END AS smoking_preference,
```

```
    COUNT(*) AS reservation_count
FROM reservation_preference rp
JOIN reservation r ON r.reservationId = rp.reservationId
WHERE r.reservationStatus IN ('BOOKED', 'CHECKED_IN')
GROUP BY smoking_preference
ORDER BY reservation_count DESC, smoking_preference;
```

OUTPUT

Non-smoking|5

Smoking|1

7. How are active stays distributed across wings right now?

SQL

```
SELECT
    b.buildingName,
    w.wingCode,
    COUNT(DISTINCT sra.stayAssignmentId) AS active_stays,
    COUNT(DISTINCT srg.guestId) AS assigned_guests
FROM stay_room_assignment sra
JOIN room r ON r.roomId = sra.roomId
JOIN level l ON l.levelId = r.levelId
JOIN wing w ON w.wingId = l.wingId
JOIN building b ON b.buildingId = w.buildingId
LEFT JOIN stay_room_guest srg ON srg.stayAssignmentId =
sra.stayAssignmentId
WHERE sra.assignmentStatus IN ('RESERVED', 'OCCUPIED')
GROUP BY b.buildingName, w.wingCode
ORDER BY active_stays DESC, assigned_guests DESC, b.buildingName,
w.wingCode;
```

OUTPUT

Main Building|A|4|6

Main Building|B|2|2

8. For each account, how much has been charged, paid, and is still outstanding?

SQL

```
WITH charge_summary AS (
    SELECT
        accountId,
        SUM(amount) AS total_charges
    FROM charge
    GROUP BY accountId
),
payment_summary AS (
    SELECT
        accountId,
        SUM(amount) AS total_payments
    FROM payment
    GROUP BY accountId
)
SELECT
    a.accountId,
    a.accountName,
    COALESCE(cs.total_charges, 0) AS total_charges,
    COALESCE(ps.total_payments, 0) AS total_payments,
    COALESCE(cs.total_charges, 0) - COALESCE(ps.total_payments, 0) AS
balance_due
FROM account a
LEFT JOIN charge_summary cs ON cs.accountId = a.accountId
```

```
LEFT JOIN payment_summary ps ON ps.accountId = a.accountId
ORDER BY balance_due DESC, a.accountId;
```

OUTPUT

6|Health Networking Mixer Master Bill|2100|0|2100

5|Consulting Workshop Master Bill|585|300|285

3|Ava Chen Stay|475|200|275

4|Board Lunch Master Bill|420|200|220

1|Emma Lee Stay|237|100|137

2|Noah Kim Stay|215|80|135

9. Which charge categories contribute the largest share of total revenue?

SQL

```
SELECT
    c.chargeType,
    COUNT(*) AS charge_lines,
    ROUND(SUM(c.amount), 2) AS total_amount,
    ROUND(AVG(c.amount), 2) AS avg_charge_amount,
    ROUND(100.0 * SUM(c.amount) / (SELECT SUM(amount) FROM charge), 1)
AS pct_of_total_revenue
FROM charge c
GROUP BY c.chargeType
ORDER BY total_amount DESC, c.chargeType;
```

OUTPUT

MEETING_ROOM|3|1580.0|526.67|39.2

FOOD_BEVERAGE|2|1440.0|720.0|35.7

ROOM_RATE|3|755.0|251.67|18.7

BUSINESS_SERVICE|1|85.0|85.0|2.1

ROOM_SERVICE|2|77.0|38.5|1.9

ROOM_SURCHARGE|1|60.0|60.0|1.5

HEALTH_CLUB|1|20.0|20.0|0.5

PHONE|1|15.0|15.0|0.4

10. Which event time slots are used the most, and how often are they for dining?

SQL

```
SELECT
    eru.usageSlot,
    COUNT(*) AS room_bookings,
    COUNT(DISTINCT eru.eventId) AS events_using_slot,
    SUM(CASE WHEN eru.isEatingUsage = 1 THEN 1 ELSE 0 END) AS
dining_bookings
FROM event_room_usage eru
JOIN event e ON e.eventId = eru.eventId
GROUP BY eru.usageSlot
ORDER BY room_bookings DESC, eru.usageSlot;
```

OUTPUT

AFTERNOON|2|2|0

EVENING|1|1|1

LUNCH|1|1|1

MORNING|1|1|0